

Immutables: Fast, Functional Data Structures for R

Shawn T. O’Neil

University of North Carolina at Chapel Hill

Abstract

We present **Immutables**, an R package providing side-effect-free data structures including: catenable, indexable sequences that also serve as double-ended queues; priority queues; key-ordered sequences; and interval indices for range overlap and similar queries. Based on monoid-annotated finger trees first described by Hinze and Paterson in 2006, almost all operations are fast, either $O(\log n)$ or amortized constant time. Finally, a developer API exposes custom monoids and aggregations for extensibility. We describe the user and developer APIs, provide benchmarks against base-R and comparable implementations, and demonstrate usage with examples.

Keywords: R, **Immutables**, persistent data structures, finger trees, monoids, functional programming.

1. Introduction

The R programming language is a natural first choice for statistical analyses, but its ever-growing popularity has diversified its application to domains as broad as data serialization and parsing, (Eddelbuettel, Stokely, and Ooms 2016; Ooms 2014), user interfaces (Jia, Yao, Jiang, Li, Wang, Li, Huang, Li, Chen, and Zhang 2022), simulation (Sipos, Massingham, Jordan, and Goldman 2011), mobile computing (Zhu, de Micheaux, Mozharovskiy, and Navarro 2022), and much more (Atchison, Anderson, Berardi, Best, Firmani, German, and Linstead 2018). Data structures, or collections, are foundational building blocks for almost all computing, providing efficient data storage, retrieval, and manipulation. A variety of structures support different features and use cases. Ordinal-indexed arrays, matrices, and lists are ubiquitous in statistical applications; stacks and queues support fast insertion and removal from ends and are widely used in state space search and scheduling applications (Zhang 1999; Gautam, Prajapati, Dabhi, and Chaudhary 2015); key-ordered sequences provide fast insertion, access, and range queries (Idreos and Callaghan 2020); priority queues provide for efficient access by minimum or maximum priority value and are often used in graph algorithms (Borodin, Boyar, Larsen, and Mirmohammadi 2010). Amongst more exotic structures, interval trees store sets of ranges for overlap-like queries and are commonly used in computational geometry (Lee and Preparata 1984).

Data structure implementations generally fall in one of two classes: *mutable* or *immutable* (the latter also known as persistent, or side-effect-free). Operations on mutable structures allow the internal structure or contents to change. Operations on immutable structures, by contrast, return changed copies, preserving the original for further use. Purely functional languages like Haskell and ML endorse persistence, and while mutating data structures are generally easier to

implement and faster, the functional programming community has invested significant effort in designing fast immutable implementations for a wide variety of applications (Okasaki 2011). This immutability is familiar to R developers, who expect pass-by-value semantics and are encouraged to develop side-effect-free functions (Matloff 2011). Most R structures follow this paradigm, with the exception of index-assignment (e.g. `x[2] <- 4.2`). Even in these cases, R’s assign-by-value and copy-on-modify semantics bound mutability (`x <- y`; `x[2] <- 4.2` modifies `x` while preserving the original data in `y`).

Unfortunately, native R types do not emphasize performance; many operations require a full scan or copy of the data, making them prohibitively expensive for use in highly dynamic or large-data applications (Gillespie and Lovelace 2016). Attempts have been made to fill these gaps. One approach is to embrace mutability, taken for example by the **collapse** package which emphasizes performance for bulk operations (Krantz 2026). Keeping immutability, we previously published an implementation of efficient, persistent stack and queue structures based on functional lazy evaluation techniques (O’Neil 2015; Goel and Vitek 2019). In 2018 T. Barry surveyed the state of collection packages, identifying 11 providing set, map, stack, queue, and/or sequence structures (Barry 2018). Of these, only three implemented side-effect-free structures: **rstackdeque** (O’Neil 2015), **sets** (Meyer and Hornik 2009), and **S4Vectors** (Pagès, Lawrence, and Aboyoun 2020). The latter two are optimized for bulk operations as compared to base-R types but, like R lists and vectors, do not support fast incremental insert/remove operations. Notably, **S4Vectors** is designed for extensibility and is widely used in the Bioconductor ecosystem, including support for interval ranges via **IRanges** (Lawrence, Huber, Pagès, Aboyoun, Carlson, Gentleman, Morgan, and Carey 2013). Barry noted a need for fast, flexible, and side-effect-free data structures to support advanced use cases in the R ecosystem (Barry 2018).

Here we describe **Immutables**, which implements a remarkable data structure known as monoid-annotated 2-3 finger trees. First described by Hinze and Paterson (Hinze and Paterson 2006), these provide persistent, amortized constant-time access to ends, $O(\log n)$ search/insertion/removal, and via monoid annotation can be purposed as other specialized structures. These form the basis of the built-in `Data.Sequence` in Haskell (Paterson, Wasserman, Felgenhauer, Feuer, and Straka 2014), and are available as libraries for Clojure (Houser 2010), OCaml (OCaml Batteries Included team 2012), F# (Tranovich 2014), JavaScript (Xu 2014), Rust (Aslanov 2018), Julia (Schauer 2014), and Go (Burdick 2020).

Immutables implements four specialized structures backed by finger trees: **flexseq**, serving as indexable, splittable, and concatenable lists with end insertion/removal for stack and queue operations; **priority_queue**, associating data elements with priority values for fast min/max access by priority; **ordered_sequence**, keeping items in order by provided keys; and **interval_index**, storing items with ranges for overlap and similar queries. Core operations are implemented in C++ via **Rcpp** (Eddelbuettel and François 2011) for speed, and a developer API is provided to extend functionality via custom monoid annotation.

2. Overview

2.1. Flexible sequences

At the core of **Immutables** is the **flexseq** structure. Much like R lists, these can store elements

of any or mixed types, nest, be named, and be indexed with the familiar `[`, `[[`, and `$` operators with integer, logical, and name vectors, including replacement. Elements can be inserted, accessed, or removed from either end via `peek/pop/push` operations in amortized constant time for efficient double-ended queue behavior. Additionally, `flexseqs` allow concatenation, splitting by position, and insertions and removals by position in fast $O(\log n)$ time.

```
fs1 <- as_flexseq(letters[1:10])
fs2 <- push_back(fs1, "z")
length(fs1)           # 10
length(fs2)           # 11

fs3 <- insert_at(fs2, 8, 4.2) # Immutables structures can store mixed types
length(fs3)           # 12

fs4 <- fs1[1:5]         # flexseq with "a", "b", "c", "d", "e"

fs5 <- pop_back(fs2)
fs5$value              # "z"
fs5$remaining          # flexseq of length 10 (without "z")
```

Other implemented methods include `c()`, `as.list()`, `unlist()`, `fapply()` (analogous to `lapply()` for lists), and `print()`, mimicking `print.list()` but displaying only the first and last few elements. Native R `for`-loops do not support iterating over elements as a result of sequences’ internal structure, but `loop()` is re-exported from the **coro** package to support iterative element access.

While `flexseqs` support names, these are somewhat limited compared to named R lists. If names are used, every element must have a unique non-NULL name. Further, the usage of names degrades worst-case performance for some operations to a full $O(n)$ scan (for example, when concatenating), though front/back access remains $O(1)$ amortized in named contexts. Otherwise, all operations are $O(\log n)$ worst case, or $O(k \log n)$ when removing or accessing k elements. For performance-sensitive applications requiring named access, ordered sequences (discussed below) can be associated with character-based names as keys for guaranteed $O(\log n)$ performance.

Example: simulating complex queueing systems

Queueing systems represent one or more service queues, where arrival and service times are typically drawn from defined distributions. The dynamics of such systems are a common area of study, for example in computing average wait times (Bose 2013). While closed-form solutions for these dynamics exist in many cases, even simple policy choices such as “each request joins the currently shortest queue” can complicate analyses significantly (Eschenfeldt and Gamarnik 2018). In this example we simulate such a system to generate an empirical wait-time distribution.

We simulate `steps` time steps; at each, a new request arrives with probability `p_arrival` and joins the shorter of two queues, `qa` or `qb`. We keep track of each server’s next free time step, processing requests from the front of its queue. Each request’s wait time is calculated and collected in a running `flexseq`, which we convert to a vector and summarize after.

```

set.seed(100)
steps <- 5000
p_arrival <- 0.9
mean_service <- 1 # service times ~ rpois(1, mean_service) + 1 (real mean 2)

qa <- flexseq(); free_a <- 0
qb <- flexseq(); free_b <- 0
waits <- flexseq()

for(t in seq_len(steps)) {
  # process new arrivals into the shorter queue
  if(runif(1) < p_arrival) {
    request <- list(arrival = t)
    if(length(qa) <= length(qb)) qa <- push_back(qa, request)
    else qb <- push_back(qb, request)
  }

  # if qa is free, process the front request and record total wait time
  if(free_a <= t && length(qa) > 0) {
    nxt <- pop_front(qa)
    qa <- nxt$remaining # overwrite queue with remaining portion
    request <- nxt$value
    waits <- push_back(waits, t - request$arrival)
    free_a <- t + rpois(1, mean_service) + 1
  }

  # if qb is free, process the front request and record total wait time
  if(free_b <= t && length(qb) > 0) {
    nxt <- pop_front(qb)
    qb <- nxt$remaining # overwrite queue with remaining portion
    request <- nxt$value
    waits <- push_back(waits, t - request$arrival)
    free_b <- t + rpois(1, mean_service) + 1
  }
}

mean(unlist(waits)) # 1.487

```

This example may be extended in several ways, including tracking queue lengths, requests that migrate between queues (as in queueing networks, [Bolch, Greiner, De Meer, and Trivedi 2006](#)), or other policies such as length-weighted random queue assignment.

2.2. Priority queues

The `priority_queue` structure stores arbitrary or mixed types, much like `flexseq`, but each element is associated with a priority value. Rather than efficient front/back access, priority queues provide efficient insertion and pop/peek by minimum and maximum priority value.

Priorities may be any orderable type (such as dates), but full C++ optimization is available for primitive numeric, character, and logical priorities. Because multiple elements may share the same priority, “all” versus “first” access methods are available: `pop_all_min(q)` returns an object with two priority queues, `$elements` containing all elements with minimum priority and `$remaining` with the rest. Conversely, `pop_min(q)` returns the first entry (by insertion order) with minimum priority in `$value`, and the rest of the priority queue in `$remaining`. Symmetric `_max()` operations are available with the same performance characteristics.

```
pq1 <- as_priority_queue(letters[1:6], priorities = c(2, 1, 3, 1, 9, 7))
peek_min(pq1)                # "b"
peek_all_min(pq1)            # priority queue with "b" "d"

pq2 <- insert(pq1, "z", priority = 10)

pq3 <- pop_max(pq2)
pq3$value                    # "z"
pq3$remaining                # priority queue with 6 entries
```

Elements of priority queues are not stored in priority order; accordingly, priority queues only support indexing and replacement with `[], [[,` and `$` by name, with the same caveats as for named `flexseqs`. Priority queues and other advanced types (below) may be cast down with `as_flexseq()` and `as.list()`, optionally preserving priority-value metadata. Like `flexseq`, priority queues may be mapped with `fapply()`; in this case the called function is supplied both the stored element and associated priority, and returns a priority queue with identical priorities on transformed values. Finally, priority queues may be iteratively accessed in priority order with `loop()`, and they additionally provide a priority-aware `merge()`.

Example: best-first feature selection

Priority queues have many applications, but we’ll demonstrate applying them to the problem of feature selection in modeling: given a large number of features, only a few of which are informative, identify a small subset with good performance. We begin by constructing a matrix `X` with 25 potential feature columns, and an outcome `y` dependent only on features 3 and 7:

```
set.seed(100)

n <- 10000
p <- 25
X <- matrix(rnorm(n * p), n, p)

# response depends on features 3 and 7 only
y <- 3.5 * X[, 3] - 4 * X[, 7] + rnorm(n)
```

We’ll consider linear models using only main effects, and use Akaike’s Information Criterion (AIC) as a measure of performance (Sakamoto, Ishiguro, and Kitagawa 1986). The number of potential feature sets (2^{25}) is too large to try all possibilities. Rather than iterate them

exhaustively, we'll perform a best-first search: given a "current best" set of features, we'll add remaining features individually, evaluating models based on them as we go and storing them for potential later expansion. By keeping feature sets in a priority queue prioritized by resultant AIC, we can select the best-performing set to build on at each iteration. We thus begin with a base model using no features, and the corresponding empty feature set and AIC stored in a `priority_queue`:

```
# a model using no features, initially the "best"
model <- lm(y ~ 1)
best_features <- numeric(0)
best_aic <- AIC(model)

# store the initial model prioritized by its AIC
pq <- priority_queue()
pq <- insert(pq, best_features, priority = best_aic)
```

So long as there are feature sets available in the queue to expand on, we will do so, but limit the total number of considered models to approximately 1,000 to prevent an exhaustive search. At each iteration we'll pop the best feature set considered so far, and compute the remaining available features.

```
models_considered <- 1

while(length(pq) > 0 && models_considered < 1000) {
  # pop the current best-performing set of features
  current_best <- pop_min(pq)

  # replace the queue with the remaining (unpopped) portion
  pq <- current_best$remaining

  # extract the current best features ($value) and AIC ($priority)
  current_best_features <- current_best$value
  current_best_aic <- current_best$priority

  # features we can add to the current best for potential improvement
  available_features <- setdiff(1:ncol(X), current_best_features)
```

At this point we iterate over `available_features`, adding each to the current best set, resulting in a new model with corresponding AIC to add to the queue. If one of these performs better than the best so far we record it. Along the way we update the number of models considered.

```
# add each unused feature one at a time
for(available_feature in available_features) {
  # add it to the set of best-performing features so far
  selected_features <- c(current_best_features, available_feature)
```

```

# build a model with that updated set
model <- lm(y ~ X[ , selected_features, drop = FALSE])

# add the features to the queue prioritized by the AIC they deliver
pq <- insert(pq, selected_features, priority = AIC(model))

# if the current model is better than the best so far, record it
if(AIC(model) < best_aic) {
  best_features <- selected_features
  best_aic <- AIC(model)
}

models_considered <- models_considered + 1
}
} # close while loop

print(best_features) # 7 3 19 24 2 21

```

In this instance the best encountered model by AIC uses features 7, 3, 19, 24, 2, and 21. Potential improvements to this example include ordering features to avoid duplicated testing of permutations, more sophisticated stopping criteria based on AIC trend, and alternative prioritization values. The search history can also be inspected efficiently by growing a `flexseq` with intermediate information.

2.3. Ordered sequences

The `ordered_sequence` type associates each element with a key value and keeps elements in key order. Keys may be any orderable type, with optimizations for primitive types, and duplicate keys are kept in insertion order. Ordered sequences support efficient insert, peek, and pop by key, with “all” versions of peek and pop for duplicate-key use cases. Ordered sequences also support key counts and range queries.

```

os1 <- as_ordered_sequence(letters[1:6], keys = c(20, 10, 30, 10, 90, 70))
peek_key(os1, 10)           # "b"
peek_all_key(os1, 10)       # ordered sequence with "b" "d"
count_key(os1, 10)          # 2
elements_between(os1, 15, 85) # ordered seq with "a" "c" "f"

os2 <- insert(os1, "z", key = 100)

os3 <- pop_key(os2, 100)
os3$value                   # character "z"
os3$remaining               # ordered sequence with "a" through "f"

```

As with other structures, ordered sequences may be named (the same name caveats apply) and indexed with `[`, `[[`, and `$`. Integer indices may be used with `[` and `[[`, in which case

they refer to elements in key-sorted order. When `[]` is used to subset an ordered sequence, the return is another ordered sequence and elements are kept in their original order. A warning is produced if this is not the requested order (e.g. `os1[c(2,1)]`). Replacement indexing (`[<-` et al.) is not supported to preserve the ordering invariant.

Also supported is `fapply()`, here calling functions accepting elements and keys and returning an ordered sequence with transformed elements, `loop()` (in key order), and key-aware `merge()`. Finally, ordered sequences support flexible key-order-based queries. These include `min_key()`, `max_key()`, `key_at()`, `nearest_key()` to identify the closest existing key to a query (undefined for keys without a simple distance metric), and `lower_bound()`, returning the first element with key greater than or equal to a query key.

Example: scalar matching without replacement

Many potential uses for ordered sequences are handled well by named lists and vectors, where access-by-key is the primary functionality. In some cases we need to dynamically find, add, and remove elements by key, but as the benchmarks in section 4 show these operations are excessively slow on large base-R structures. Matching without replacement is one such case, often employed for cohort matching purposes in observational studies using propensity scores (Benedetto, Head, Angelini, and Blackstone 2018). Forgoing details, “treated” patients are matched to a subset of distinct “control” patients with similar scores. We start by simulating some populations and scores, and initialize an empty `flexseq` to store matches. We also convert the treated and untreated row numbers (serving as identifiers) to ordered sequences, keyed by their score.

```
set.seed(100)

n_patients <- 200
patients <- data.frame(treated = sample(c(TRUE, FALSE),
                                       n_patients,
                                       prob = c(0.1, 0.9),
                                       replace = TRUE),
                      score = runif(n_patients))

# row numbers act as identifiers
treated_rows <- which(patients$treated)
treated_scores <- patients$score[patients$treated]

untreated_rows <- which(!patients$treated)
untreated_scores <- patients$score[!patients$treated]

matches <- flexseq()

treated_seq <- as_ordered_sequence(treated_rows, keys = treated_scores)
untreated_seq <- as_ordered_sequence(untreated_rows, keys = untreated_scores)
```

Matching itself is a simple greedy selection. Popping treated patients in order, each is matched to their nearest remaining untreated record by score key. That patient is popped from wher-

ever it was located, and a data frame describing the match is pushed onto the accumulating `flexseq`. This example also illustrates collecting these into a single result data frame for use after.

```
while(length(treated_seq) > 0) {
  # no untreated patients left to match against
  if (length(untreated_seq) == 0L) break

  # pop the first (lowest-score) treated patient
  front_el <- pop_front(treated_seq)
  treated_seq <- front_el$remaining

  treated_pt_score <- front_el$key
  treated_pt_row <- front_el$value

  # find and remove the nearest-score untreated patient
  match_key <- nearest_key(untreated_seq, treated_pt_score)
  match_el <- pop_key(untreated_seq, match_key)
  untreated_seq <- match_el$remaining

  untreated_pt_score <- match_el$key
  untreated_pt_row <- match_el$value

  match_row <- data.frame(treated_pt_row,
                          treated_pt_score,
                          untreated_pt_row,
                          untreated_pt_score)

  matches <- push_back(matches, match_row)
}

match_df <- do.call(rbind, as.list(matches))
```

2.4. Interval indices

Like other structures above, the `interval_index` type stores arbitrary elements with list-like semantics, but each is associated with a $[start, end)$ range. Interval indices thus support peeking and popping by query point, as well as query interval overlap, containment, and subsumption. Each of these also has ‘first’ and ‘all’ return variants; `pop_overlapping(ix, start, end)` removes and returns the first (in start order, breaking ties by insertion order) overlapping range in `ix`, while `pop_all_overlapping(ix, start, end)` removes and returns all. By default, all bounds are treated as left-closed (`[]`); this can be adjusted at initialization time, or per-query by supplying a `bounds` argument, in which case the interpretation is applied to both the query and stored intervals. Interval endpoints are particularly important for sweep-line algorithms, so point-based queries can be limited to matching over the whole interval or only at one or both endpoints.

```

ix1 <- as_interval_index(c("a", "b", "c"),
                        start = c(1, 2, 4),
                        end = c(3, 4, 5))

peek_point(ix1, 3) # character "b"

# start-only query; can also query at "end" or "either"
peek_point(ix1, 2, match_at = "start") # character "b" only

ix2 <- insert(ix1, "d", start = 5, end = 6)

ix3 <- pop_all_overlapping(ix2, 3, 4.5)
ix3$elements      # interval index with "b" and "c"
ix3$remaining     # interval index with "a" and "d"

```

Beyond `fapply()`, `merge()`, `loop()`, and conversion methods, interval indices can also report global minimum and maximum range endpoints.

Example: hospital occupancy over time

Many datasets are characterized by intervals, such that we wish to query which intervals overlap a given point. Sweep-line algorithms do so over points in order. Here we illustrate this basic pattern, considering a hypothetical computation of hospital occupancy over time. Patients arrive randomly, stay for a random number of days, and are associated with demographic variables for age and sex. For each day, we wish to capture the total occupancy, average age, and percent male. We begin by simulating data and building an interval index storing each patient with their stay range.

```

set.seed(100)

num_days <- 365
n_patients <- 500

starts <- sample(1:num_days,
                size = n_patients,
                replace = TRUE)

ends <- starts + rpois(n_patients, 2) # average stay = 2 days

ages <- pmax(0, rnorm(n_patients, mean = 35, sd = 10))
is_males <- sample(c(0, 1), n_patients, replace = TRUE)

# list of patients (each a list with age and is_male)
patients <- Map(list, age = ages, is_male = is_males)

# store each patient in an interval index with their stay
ix <- as_interval_index(patients, start = starts, end = ends)

```

Next, for each day we'll query for overlapping patients, computing summary statistics. These are collected in a `flexseq`, which we later convert to a data frame for further analysis. This example also illustrates using `fapply()` to unpack the contents of an interval index containing named lists.

```
day_stats <- flexseq()

for(day in seq_len(num_days)) {
  present <- peek_all_point(ix, day)
  occupancy <- length(present)

  if(occupancy == 0) {
    stats <- data.frame(day = day, occupancy = 0, mean_age = NA, pct_male = NA)
  } else {
    day_ages <- present |>
      fapply(function(patient, start, end) {
        patient$age
      }) |>
      unlist() # fapply returns an interval_index, convert to vector

    # use fapply() on the interval index to map each patient record
    # (containing both $age and $is_male) to an interval_index
    # containing just is_male values
    day_males <- present |>
      fapply(function(patient, start, end) {
        patient$is_male
      }) |>
      unlist()

    stats <- data.frame(day = day,
                        occupancy = occupancy,
                        mean_age = mean(day_ages),
                        pct_male = 100 * mean(day_males))
  }

  day_stats <- push_back(day_stats, stats)
}

# convert the flexseq of data frames into a single data frame
# with columns for day, occupancy, mean_age, and pct_male
stats_df <- do.call(rbind, as.list(day_stats))
head(stats_df)
```

This example is designed for simplicity, but is not optimally efficient. Real sweep-line algorithms consider only used range endpoints, and keep three indices: one with as-yet-unprocessed elements, an active set overlapping the virtual sweep line, and a processed set, moving elements through them with `pop_all_point()`.

3. Developer API

Structures in **Immutables** are backed by monoid-annotated finger trees as originally described by Hinze and Paterson (Hinze and Paterson 2006). They are simultaneously persistent, performant, and flexible, supporting a variety of container types and operations. **Immutables** provides a developer API exposing this functionality. Here we describe the API and implement two additional structures: first, a simple accumulator over task times to introduce the API components, and second, a subsequence-search structure over DNA strings to illustrate their flexibility.

In functional programming, a *monoid* pairs an associative binary function with an identity element, for example $(+, 0)$. In the parlance of Hinze and Paterson, a *measure* is a function mapping stored data elements to values operated on by a monoid. For our accumulator example, we begin with a base-R list of tasks, each with an `id` and estimated number of required hours to complete:

```
set.seed(42)

# flexseq of lists, each with $id and $hours
tasks <- lapply(1:11,
  function(i) {
    list(id = letters[i],
         hours = rpois(n = 1, lambda = 5))
  }) |>
  as_flexseq()
```

Given a single entry `el`, a measuring function `function(el) el$hours` can be used to map tasks to their estimated completion time, and when paired with the monoid $(+, 0)$ produces total time across any sequence of tasks. The `measure_monoid()` function collects these pieces together:

```
total_time_mm <- measure_monoid(`+`, 0, function(el) el$hours)
```

This monoid can then be attached to the `tasks` sequence with `add_monoids()`. Structures can be associated with multiple monoids, so they are provided as a named list:

```
tasks2 <- add_monoids(tasks, list(total_time = total_time_mm))
```

Internally, `flexseqs` are tree structures with user values stored in leaf nodes. Their operations are supported by a set of default monoids; each of `priority_queue`, `ordered_sequence`, and `interval_index` subclass `flexseq` and add additional monoids supporting their unique operations. The `plot_structure()` function shows the internal structure, where node labels can be specified as a function of nodes. Each node has a `$type` set to "Element" for nodes storing user data, these also have an `$element` storing the data itself, and all nodes carry a `$measures` storing monoid information. The following code illustrates our annotated tasks sequence in Figure 1.

```

plot_structure(tasks2, node_label = function(node) {
  if(node$type == "Element") {
    sprintf("ID: %s, H: %d\nTotal: %d",
            node$element$id,
            node$element$hours,
            node$measures$total_time)
  } else {
    sprintf("Total: %d", node$measures$total_time)
  }
})

```

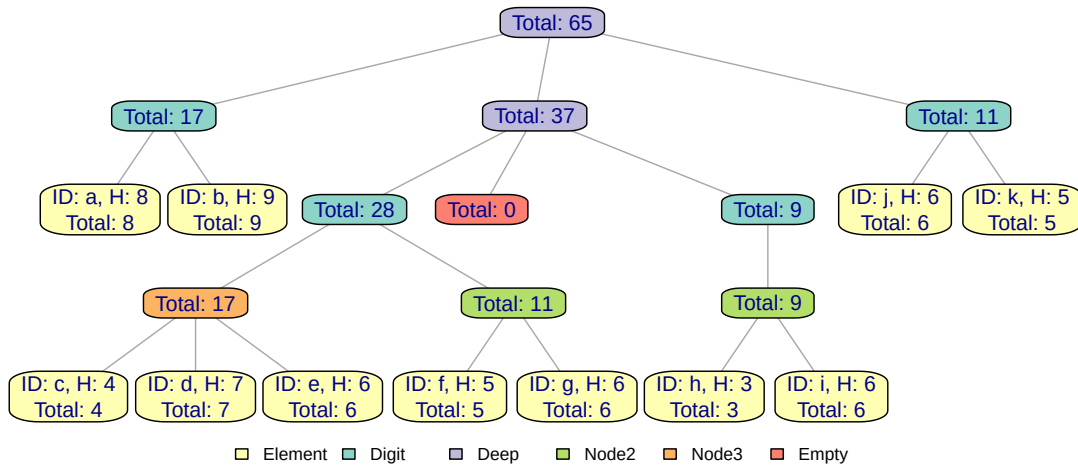


Figure 1: Internal structure of a monoid-annotated finger tree. User values are stored at leaves, illustrated with ID and H (for Hours). Leaves and internal nodes accumulate a `total_time` (shown as `Total`) measure using the associated monoid.

Several features of Figure 1 are notable. First, data elements are stored in leaves, in the order of construction. Custom orders are possible via monoid-based operations; `ordered_sequence` implements these and provides a convenient key-ordered variant. Second, each leaf node also carries its measure, here the `$hours` value as defined in `total_time_mm`. Third, internal nodes have different types: `Deep`, `Single`, and so on. Finally, internal nodes also have a measure value, produced by applying the monoid function `+` to all children’s measures. These values are accumulated beginning with the monoid identity `0`, and because `+` is associative measures can be accurately aggregated and cached recursively. Understanding these internals is not required for developer use, but this complex structure ensures tree balance for $O(\log n)$ operations overall, and that first and last elements are a maximum of two levels deep for amortized $O(1)$ operations on ends. (Time bounds are amortized because some additions trigger a cascade of restructuring, but over n insertions the runtime is guaranteed $O(n)$.)

The final piece of the monoid-annotation puzzle comes in the form of a binary *predicate* function, which must be *monotone over the sequence of accumulated measure values on leaves*. To motivate, suppose we wish to identify which tasks we can fully complete in a standard 40-hour work week; the answer is tasks `a` through `f` totaling 39 hours. The following code

splits `tasks2` into two subsequences: a `$left` whose tasks sum to less than or equal to 40 hours (which may be empty), and a `$right` with the remainder (which also may be empty).

```
split_tasks <- split_by_predicate(tasks2,
                                predicate = function(m) m > 40,
                                monoid_name = "total_time")

split_tasks$left  # flexseq with tasks a through f
split_tasks$right # flexseq with tasks g through k
```

Functionally, `split_by_predicate()` accumulates measure values defined by the `total_time` monoid $(+, 0)$; `a` is associated with $0 + 8$, `b` with $(0 + 8) + 9$, `c` with $((0 + 8) + 9) + 4$, and so on. The input sequence is split the first time the provided predicate over these resolves to `TRUE`. Notice that internal node totals are not used in this accumulation; rather, they are used by `split_by_predicate()` to identify the split point with a single efficient root-to-leaf traversal. Monotonicity of the predicate (switching from `FALSE` to `TRUE` at most once) is thus necessary for correctness.

3.1. Example: subsequence search

Here we'll illustrate how these components can be used to build an efficient subsequence-search data structure modeled after a suffix array, a common approach to finding exact matches in large DNA strings.

```
sequence <- "ACGCGCTCGCGCATAGTCGCGCCTG"
query    <- "CGCGC" # goal: find indices where query occurs (2, 8, and 18)
```

For simplicity we assume that query strings are at most 8 characters long, and store the position of length-8 subsequences in an `ordered_sequence` keyed on the subsequence: key `ACGCGCTC` will be associated with index 1, `CGCGCTCG` with index 2, and so on. However, the ordered sequence will store items lexicographically by key: the first few elements will be `ACGCGCTC` at index 1, `AGTCGCGC` at 15, `ATAGTCGC` at 13, and so on. Consequently, query matches will be found as prefixes of keys in a continuous run.

In order to find the endpoints of a matching run we annotate the sequence with a monoid that tracks the *maximum* key value, with associative function `max` and identity value `-Inf`. When used with an ordered sequence, the measuring function is passed a list containing both the stored `$value` (here the index) and a `$key`. To begin, we'll attach this measuring monoid to an empty ordered sequence, and then fill it with keyed values; the structure will ensure the custom monoid is applied as elements are added.

```
# max-key monoid
max_seq <- measure_monoid(max, i = -Inf, measure = function(entry) entry$key)

# add the monoid to an empty ordered sequence
subseqs <- ordered_sequence()
subseqs <- add_monoids(subseqs, list(max_seq = max_seq))
```

```
# fill with indices keyed on length-8 substrings
for(index in 1:nchar(sequence)) {
  subseq <- substr(sequence, index, index + 7)
  subseqs <- insert(subseqs, index, key = subseq)
}
```

With the structure built, we can find the range matching the query. First we split the sequence using a predicate that triggers on the first key that is $\geq$ the query. If any is found `$right` will be non-empty, and the first entry in it *may* have a prefix that is an exact match for the query, indicating a hit. In this case we compute the measure of the first entry using `get_measures()`, which returns a `flexseq` of measure values given a measure name; subsetting `$right` to the first entry prior ensures this result is not large. If the first entry matches, we can use a second predicate to find the first entry that is greater than the query and is not an exact prefix match, capturing in `$left` all exact-match locations. At the end we use `fapply()` to extract only the matching indices as a vector.

```
matches <- ordered_sequence()

first_split <- split_by_predicate(subseqs, function(m) query <= m, "max_seq")

if(length(first_split$right) > 0) {
  first_entry <- first_split$right[1]

  # get_measures() returns a flexseq of measure values, [[1]] extracts
  # the first (and only) returning a bare measure value
  first_measure = get_measures(first_entry, "max_seq")[[1]]

  # if the query is found in the first measure, we need to find the
  # first entry with a key larger than the query that does not match it
  if(startsWith(first_measure, query)) {
    second_split <- split_by_predicate(
      first_split$right,
      function(m) query < m & !startsWith(m, query),
      "max_seq"
    )
    matches <- second_split$left
  }
}

positions <- fapply(matches, function(value, key) value) |> unlist()
positions # reports [1] 8 18 2
```

While this structure will not outperform optimized suffix-array bioinformatics tools which can achieve linear $O(n)$ build time (Puglisi, Smyth, and Turpin 2007), the performance characteristics here are strong, with an $O(n \log n)$ build time and $O(k + \log n)$ query time where k is the number of matches.

4. Benchmarks

As discussed above, few comparable side-effect-free structures are implemented in R for comparison. Barry compared performance across several non-persistent structures in addition to persistent, naïve implementations using base-R lists, which are less performant than vectors but support complex element types (Barry 2018). Here we benchmark **Immutableables** structures against similar baselines, and for interval indices also compare against **IRanges**, a popular persistent interval-range data structure. All operations were timed with `bench::mark()` with 10 iterations on newly initialized structures, garbage collecting with `gc()` before each. Because operations dispatched through S3 methods incur first-call costs (method resolution, byte-compilation) that dominate timing at small n , we warm each operation with three untimed calls before measurement. Full benchmark code and results can be found in supplementary materials and in the *Benchmarks* package vignette.

Sequence and queue operations benchmarked include enqueue (via `push_back()`), dequeue (`pop_front()`), concatenation (`c()`), and operations around the middle element: replacement (`[[<-`), removal (`pop_at()`), and splitting (`split_at()`). Base-R sequences were implemented as lists, with index-based access, removal via negative indexing (e.g. `1[-1]` for dequeue), and concatenation and enqueue backed by `c()`. To reveal scaling dynamics we ran these tests on structures of between 2^{12} (4,096) and 2^{18} (262,144) elements. Results in Figure 2 reveal the $O(n)$ nature of operations on base-R lists. The $O(\log n)$ sequence operations outperform base lists at between 2^{13} (for enqueue/dequeue) and 2^{17} (for middle-element operations) elements.

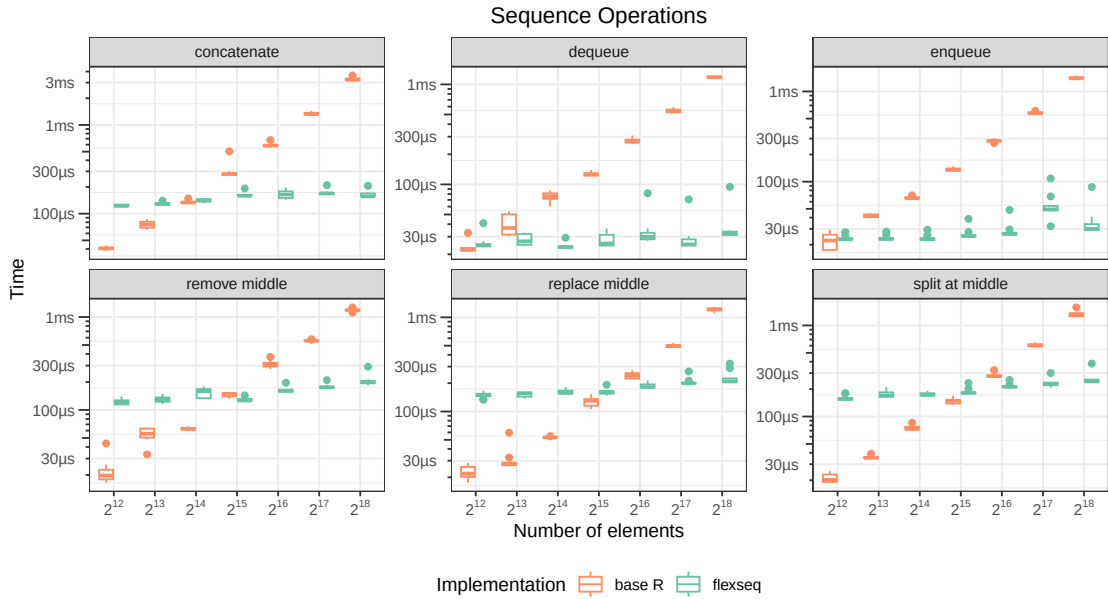


Figure 2: Benchmark timings for sequence operations.

Ordered sequences keep elements sorted by a key value, where duplicate keys are allowed. Like `flexseq`, access to ends (minimum and maximum key) is amortized $O(1)$, and other operations are $O(\log n)$. In addition to basic insert/peek/pop operations, we benchmarked count key, key range query, and lower bounding, a common building block supporting other operations such as `nearest_key()`. Base-R element-storing lists were paired with an integer

key vector and sorted once by key; lookups used `match()`, and counts and range queries used logical comparisons over the key vector. All operations were based on the median key, a worst-case scenario for our ordered sequences. Results in Figure 3 show that base-R `match()`-based lookup and logical indexing is highly optimized, achieving $\sim 200\mu\text{s}$ key lookups for 2^{18} elements. Nevertheless, the linear / logarithmic divide between implementations is evident, and ordered sequences outperform base-R lookup at between 2^{17} (131K) and 2^{20} (1M) elements, with the exception of `peek_key()`, where ordered sequences outperform at 2^{23} (8.4M) elements (not shown).

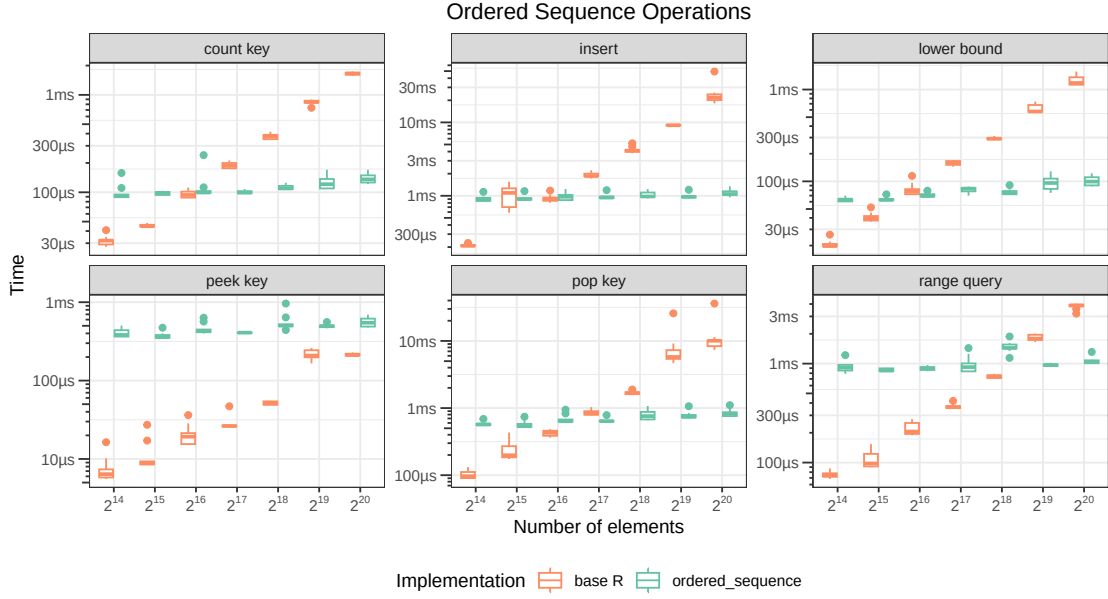


Figure 3: Benchmark timings for ordered sequence operations.

For priority queues we tested simple insertion, peeking, and popping the maximum priority element (we did not test symmetric `min`-based operations which have identical performance characteristics). Base-R priority queues were implemented on an R list with corresponding numeric priority vector, using `which.max()` for index identification and negative `[]`-based indexing and removal. Priorities were randomly generated with `runif()` with default parameters. Figure 4 reveals priority queues to be substantially faster than the naïve implementation at 2^{17} elements and beyond.

We compared interval indices against both base-R and Bioconductor’s **IRanges** implementations, which are also persistent. In both, values were stored in a list parallel to the ranges. Base-R ranges were stored in a data frame with `start` and `end` columns, queried by logical selection over the columns and `[]`-indexing, with insertion via `rbind()`. **IRanges** insertion used `c()`, and queries were resolved with `IRanges::findOverlaps()`, extracting matching positions with `S4Vectors::subjectHits()` (or `S4Vectors::queryHits()` for within queries). Removal by overlap took the `setdiff()` of all positions and the overlapping hits, then indexed both the values list and the **IRanges** object with the remaining positions.

Intervals were generated with integer endpoints, as required by **IRanges**: n start coordinates were sampled without replacement from $[1, 10n]$ and sorted, each paired with a width drawn uniformly from $[1, 100]$, yielding a sparse collection of short ranges that overlap their near

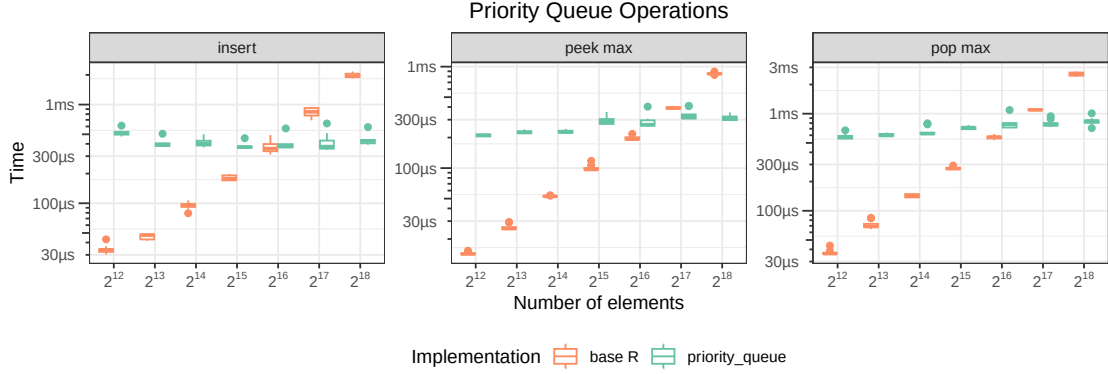


Figure 4: Benchmark timings for priority queue operations.

neighbors. We tested insertion of a new interval, point-based queries for first and all overlapping intervals, range-based overlap and containment queries, and removal of all intervals overlapping a query range. Query points were taken from the median stored start position, and query windows extended from this median start to the median + 50th start location, resulting in ~ 50 overlaps for overlap operations.

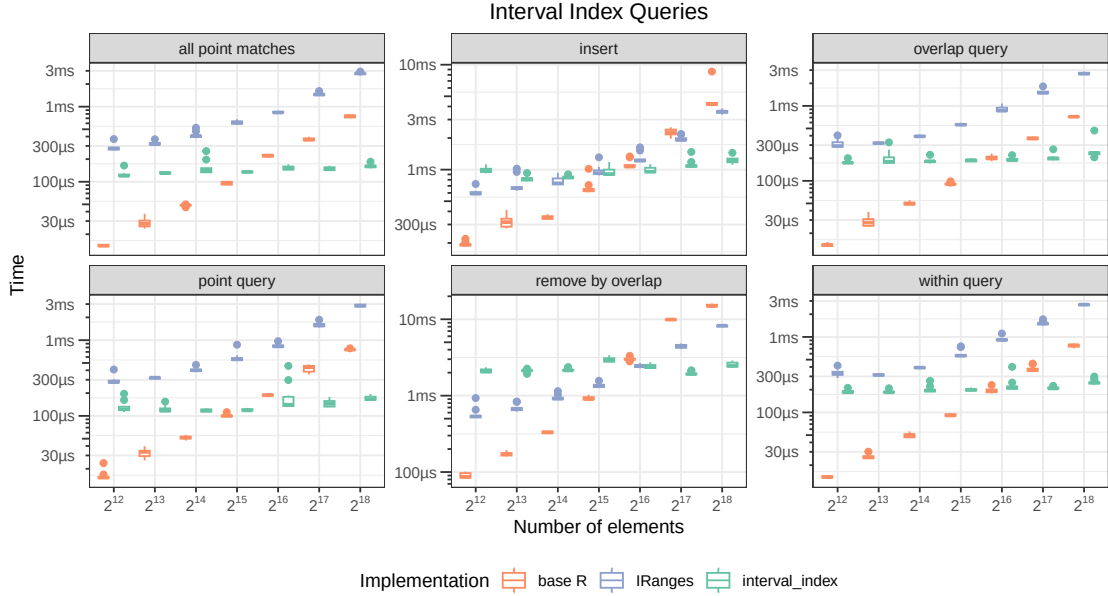


Figure 5: Benchmark timings for interval index operations.

Results in Figure 5 show **IRanges** to be faster than base-R for insertion and removal operations, but slower for read-only queries. Both implementations exhibit $O(n)$ growth, and interval indices outperform both at between 2^{16} elements for read operations and 2^{17} elements for insertions and removals.

5. Conclusion

R’s continued growth in popularity across many application domains is in part due to its multi-paradigm nature. Object orientation is provided by multiple class systems, including **R6** which formalizes first-class support for mutable data types. Loops and **if/then/else** constructs are familiar to those coming from more procedural languages. Still, R’s roots as a functional language dominate: functions are pass-by-value, and the generic-dispatch **S3** object system is most commonly used, followed by **S4** in the Bioconductor community (Gentleman, Carey, Bates, Bolstad, Dettling, Dudoit, Ellis, Gautier, Ge, Gentry, Hornik, Hothorn, Huber, Iacus, Irizarry, Leisch, Li, Maechler, Rossini, Sawitzki, Smith, Smyth, Tierney, Yang, and Zhang 2004); both are typically used in a value-oriented style, where updates create new objects via copy-on-modify semantics rather than in-place mutation. Seasoned R programmers avoid dynamically growing or shrinking structures, which can easily result in $O(n^2)$ behavior.

The relative lack of performant, immutable collection types in R has been previously noted, and Barry (2018) calls specifically for their development. **Immutables** addresses this by drawing on existing methods in purely functional data structures, providing fast off-the-shelf implementations of several common data structures, as well as a developer API for custom structures via monoid-annotation.

Computational details

Immutables is available on CRAN at <https://CRAN.R-project.org/package=Immutables> and may be installed with `install.packages("Immutables")`. All source code is available at <https://github.com/oneilsh/Immutables>, including a `replication_code.R` script in the `data-raw/replication` directory. User-facing documentation is available in package help pages and vignettes, with **pkgdown**-formatted documentation available at <https://oneilsh.github.io/Immutables>.

The results in this paper were obtained using R version 4.5.3 (2026-03-11) on aarch64-apple-darwin20 with **Immutables**~1.2.0 and the comparison packages **IRanges**~2.44.0 and **S4Vectors**~0.48.1. Tests were performed on an Apple M3 Max running macOS 26.6.2.

References

- Aslanov P (2018). “fingertrees: Immutable Persistent Finger Trees for Rust.” <https://github.com/aslpavel/fingertree-rs>. Crate: <https://crates.io/crates/fingertrees>, version 0.2.11. Accessed 2026-09-25.
- Atchison A, Anderson H, Berardi C, Best N, Firmani C, German R, Linstead E (2018). “A topic analysis of the R programming language.” In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*, pp. 183–184. ACM, New York, NY, USA. doi:10.1145/3183440.3195087. URL <http://dx.doi.org/10.1145/3183440.3195087>.
- Barry T (2018). “Collections in R: Review and Proposal.” <https://digitalcommons.unl.edu/r-journal/624>. Accessed: 2026-6-11.

- Benedetto U, Head SJ, Angelini GD, Blackstone EH (2018). “Statistical primer: propensity score matching and its alternatives.” *European Journal of Cardio-Thoracic Surgery*, **53**(6), 1112–1117.
- Bolch G, Greiner S, De Meer H, Trivedi KS (2006). *Queueing networks and Markov chains: modeling and performance evaluation with computer science applications*. John Wiley & Sons.
- Borodin A, Boyar J, Larsen KS, Mirmohammadi N (2010). “Priority algorithms for graph optimization problems.” *Theor. Comput. Sci.*, **411**(1), 239–258. doi:10.1016/j.tcs.2009.09.033. URL <http://dx.doi.org/10.1016/j.tcs.2009.09.033>.
- Bose SK (2013). *An introduction to queueing systems*. Springer Science & Business Media.
- Burdick Jr WR (2020). “go-fingertree: Finger Trees in Go.” <https://github.com/zot/go-fingertree>. Port of fingertree.js. Accessed 2026-09-25.
- Eddelbuettel D, François R (2011). “Rcpp: Seamless R++Integration.” *J. Stat. Softw.*, **40**(8), 1–18. doi:10.18637/jss.v040.i08. URL <https://doi.org/10.18637/jss.v040.i08>.
- Eddelbuettel D, Stokely M, Ooms J (2016). “RProtoBuf: Efficient Cross-Language Data Serialization in R.” *J. Stat. Softw.*, **71**(2), 1–24. doi:10.18637/jss.v071.i02. URL <https://doi.org/10.18637/jss.v071.i02>.
- Eschenfeldt P, Gamarnik D (2018). “Join the shortest queue with many servers. The heavy-traffic asymptotics.” *Mathematics of Operations Research*, **43**(3), 867–886.
- Gautam JV, Prajapati HB, Dabhi VK, Chaudhary S (2015). “A survey on job scheduling algorithms in Big data processing.” In *2015 IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT)*, pp. 1–11. IEEE. doi:10.1109/icecct.2015.7226035. URL <http://dx.doi.org/10.1109/ICECCT.2015.7226035>.
- Gentleman RC, Carey VJ, Bates DM, Bolstad B, Dettling M, Dudoit S, Ellis B, Gautier L, Ge Y, Gentry J, Hornik K, Hothorn T, Huber W, Iacus S, Irizarry R, Leisch F, Li C, Maechler M, Rossini AJ, Sawitzki G, Smith C, Smyth G, Tierney L, Yang JYH, Zhang J (2004). “Bioconductor: open software development for computational biology and bioinformatics.” *Genome Biol.*, **5**(10), R80. doi:10.1186/gb-2004-5-10-r80. URL <http://dx.doi.org/10.1186/gb-2004-5-10-r80>.
- Gillespie C, Lovelace R (2016). *Efficient R Programming*. O’Reilly Media, Inc. URL <https://www.oreilly.com/library/view/efficient-r-programming/9781491950777/>.
- Goel A, Vitek J (2019). “On the design, implementation, and use of laziness in R.” *Proc. ACM Program. Lang.*, **3**(OOPSLA), 1–27. doi:10.1145/3360579. URL <http://dx.doi.org/10.1145/3360579>.
- Hinze R, Paterson R (2006). “Finger trees: a simple general-purpose data structure.” *J. Funct. Prog.*, **16**(2), 197–217. doi:10.1017/s0956796805005769. URL <http://dx.doi.org/10.1017/S0956796805005769>.

- Houser C (2010). “data.finger-tree: Persistent Collections Based on 2-3 Finger Trees.” Clojure contrib library, <https://github.com/clojure/data.finger-tree>. Version 0.1.1 (Maven: org.clojure/data.finger-tree). Accessed 2026-09-25.
- Idreos S, Callaghan M (2020). “Key-value storage engines.” In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, New York, NY, USA. doi:10.1145/3318464.3383133. URL <http://dx.doi.org/10.1145/3318464.3383133>.
- Jia L, Yao W, Jiang Y, Li Y, Wang Z, Li H, Huang F, Li J, Chen T, Zhang H (2022). “Development of interactive biological web applications with R/Shiny.” *Brief. Bioinform.*, **23**(1), bbab415. doi:10.1093/bib/bbab415. URL <https://dx.doi.org/10.1093/bib/bbab415>.
- Krantz S (2026). “Collapse: advanced and fast statistical computing and data transformation in R.” *Journal of Statistical Software*, **116**, 1–38.
- Lawrence M, Huber W, Pagès H, Aboyoun P, Carlson M, Gentleman R, Morgan MT, Carey VJ (2013). “Software for computing and annotating genomic ranges.” *PLoS Comput. Biol.*, **9**(8), e1003118. doi:10.1371/journal.pcbi.1003118. URL <http://dx.doi.org/10.1371/journal.pcbi.1003118>.
- Lee, Preparata (1984). “Computational geometry—A survey.” *IEEE Trans. Comput.*, **C-33**(12), 1072–1101. doi:10.1109/tc.1984.1676388. URL <http://dx.doi.org/10.1109/TC.1984.1676388>.
- Matloff N (2011). *The art of R programming: A tour of statistical software design*. No Starch Press, San Francisco, CA. URL <https://diytranscriptomics.com/Reading/files/The%20Art%20of%20R%20Programming.pdf>.
- Meyer D, Hornik K (2009). “Generalized and Customizable Sets in R.” *J. Stat. Softw.*, **31**(2), 1–27. doi:10.18637/jss.v031.i02. URL <https://doi.org/10.18637/jss.v031.i02>.
- OCaml Batteries Included team (2012). “BatFingerTree: Generic Finger Trees, module of OCaml Batteries Included.” <https://github.com/ocaml-batteries-team/batteries-included>. API documentation: <https://ocaml.org/p/batteries/latest/doc/index.html>. Accessed 2026-09-25.
- Okasaki C (2011). *Purely Functional Data Structures*. Cambridge University Press, Cambridge, England. doi:10.5555/925926. URL <http://dx.doi.org/10.5555/925926>.
- O’Neil Shawn t (2015). “Implementing persistent O(1) stacks and queues in R.” *R J.*, **7**(1), 118. doi:10.32614/rj-2015-009. URL <https://doi.org/10.32614/RJ-2015-009>.
- Ooms J (2014). “The jsonlite package: A practical and consistent mapping between JSON data and R objects.” *arXiv [stat.CO]*. doi:10.48550/arXiv.1403.2805. URL <http://arxiv.org/abs/1403.2805>.
- Pagès H, Lawrence M, Aboyoun P (2020). “S4Vectors: Foundation of vector-like and list-like containers in Bioconductor.” *R package version 0.26*, **1**.

- Paterson R, Wasserman L, Felgenhauer B, Feuer D, Straka M (2014). “Data.Sequence: Finite Sequences, module of the containers package.” Haskell package `containers`, version 0.8, <https://hackage-content.haskell.org/package/containers-0.8/docs/Data-Sequence.html>. Implements 2-3 finger trees annotated with sizes, following section 4.2 of ?. Maintainer: `libraries@haskell.org`. Accessed 2026-09-25.
- Puglisi SJ, Smyth WF, Turpin AH (2007). “A taxonomy of suffix array construction algorithms.” *ACM Comput. Surv.*, **39**(2), 4. doi:10.1145/1242471.1242472. URL <http://dx.doi.org/10.1145/1242471.1242472>.
- Sakamoto Y, Ishiguro M, Kitagawa G (1986). “Akaike information criterion statistics.” *Dordrecht, The Netherlands: D. Reidel*, **81**(10.5555), 26853.
- Schauer M (2014). “FingerTrees.jl: The Persistent Data Structure Finger Tree in Julia.” <https://github.com/mschauer/FingerTrees.jl>. Version 0.1.3. Accessed 2026-09-25.
- Sipos B, Massingham T, Jordan GE, Goldman N (2011). “PhyloSim - Monte Carlo simulation of sequence evolution in the R statistical computing environment.” *BMC Bioinformatics*, **12**(1), 104. doi:10.1186/1471-2105-12-104. URL <http://dx.doi.org/10.1186/1471-2105-12-104>.
- Tranovich J (2014). “fsharp-finger-trees: Finger Trees in F#.” <https://github.com/cantsin/fsharp-finger-trees>. Accessed 2026-09-25.
- Xu X (2014). “fingertree.js: Implementation of Finger Tree in JavaScript.” <https://github.com/qiao/fingertree.js>. Npm package: `fingertree`. Accessed 2026-09-25.
- Zhang W (1999). *State-space search: Algorithms, complexity, extensions, and applications*. Springer Science & Business Media.
- Zhu A, de Micheaux PL, Mozharovskiy P, Navarro F (2022). “R on Raspberry Pi: The “RaspberryPiR” package for collecting and analysing streaming sensor data.” In *useR! 2022*. URL <https://hal.science/hal-05522032>.

Affiliation:

Shawn T. O’Neil

University of North Carolina at Chapel Hill

Department of Genetics 120 Mason Farm Road 5000 D, Genetic Medicine Building CB#7264

UNC-Chapel Hill Chapel Hill, NC 27599-7264

E-mail: shawn@tislab.org